指令设计与功能验证

西安电子科技大学　香山芯锐

从指令规格解释八路乘法与两拍归约，展示 RTL 接入、正负数及边界测试，再检查代表性内部时序。

赛题 2.1、2.2、2.3.1～2.3.3 | 实验材料截至 2026-10-30 | 第五版 · 2026-11-04

4 指令规格与微架构设计

VDot 单元放在香山向量后端的 VFEX0 中。收到两个源寄存器后，它取出各自低八个字节，按有符号数逐项相乘，再把乘积相加。结果写进目的寄存器的低 32 位，高 96 位清零。单元用两拍完成乘法、求和和结果整理；软件只需发出一条 `vdot.vv`。

VDot 接入原有向量后端：重命名确定要读写哪几个物理寄存器，发射队列等输入就绪后送出指令，结果沿现有写回通路返回，ROB 最后按程序顺序退休。新增硬件负责点积计算，并向后端报告非法配置等异常。

本章对应赛题 2.1，依次说明软件可见行为、位宽选择、流水划分和状态处理。第 5 章给出这些设计在 Chisel 及整核接口中的具体实现。

4.1 固定配置与 custom-0 编码

指令采用 custom-0 编码，当前计算配置固定为 SEW=8、LMUL=1、VLEN=128。

处理器采用昆明湖 V2 单核 MinimalConfig，向量宽度为 128 位，ROB 为 48 槽。赛题要求固定八项 signed INT8、归约初值为零和其余输出清零。项目据此定义结果布局、掩码、合法 VL 及状态更新，作为 RTL 与参考模型共同遵循的实现规格。

无掩码形式写为 `vdot.vv vd,vs2,vs1`，掩码形式追加 `v0.t`。当前工具链通过 `.word` 发出 custom-0 机器码。

表 4-1　指令编码

位段	字段	取值或含义
[31:26]	funct6	`000000`
[25]	vm	1：全部项参与；0：由旧 v0 的低八位使能
[24:20]	vs2	第二向量源
[19:15]	vs1	第一向量源
[14:12]	funct3	`000`
[11:7]	vd	向量目的
[6:0]	opcode	`0001011`，即 custom-0 `0x0b`

识别条件为 `(word & 0xfc00707f) == 0x0000000b`。例如 `vdot.vv v3,v2,v1` 编码为 `0x0220818b`，相应的掩码形式为 `0x0020818b`。保存的编码检查枚举了 65,536 种寄存器及 vm 组合，在锁定版本的香山主译码表与 NEMU 顶层规则中均未发现交叠〔1〕。

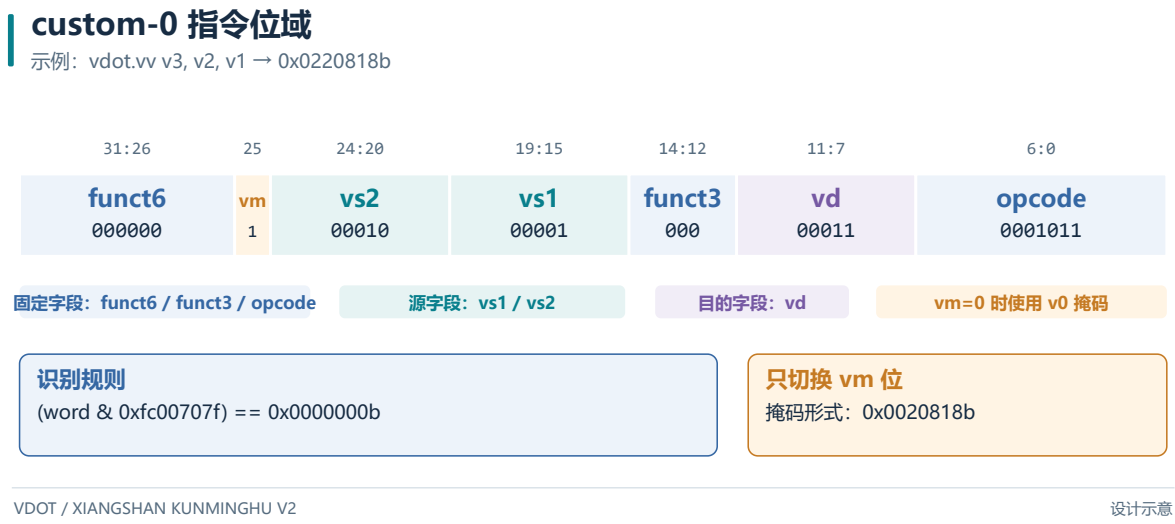


图 4-1　32 位 custom-0 编码。字段宽度按位数绘制；vm、两个源和目的寄存器占 16 个可变位。

4.2 运算语义、掩码与结果布局

两个源寄存器都从最低字节开始编号，最低字节是第 0 项。取第 i 个字节时，按 8 位补码解释为 a_i、b_i，例如 `0x80` 表示 −128。`vm=1` 时八项全部参与；`vm=0` 时，第 i 项是否参与由旧 v0 的 bit i 决定，写成公式就是 m_i = `vm OR old_v0[i]`：

$$S = \Sigma_{i=0..7}\ m_i \times a_i \times b_i$$

```
vd[31:0]   = sign_extend_32(S)
vd[127:32] = 0
```

掩码只决定某项乘积要不要加入总和，不把旧目的值加进去。计算固定读取源寄存器低 64 位的八个字节，VL=16 时也一样。合法非零 VL 执行完后，目的寄存器的低 32 位是点积，高 96 位全部为零。

零掩码表示八项都不参与求和，因此结果为零，整个目的寄存器被清零。VL=0 的要求不同：保留目的寄存器原来的全部 128 位。硬件因此既需要计算点积，也需要把 oldVd，也就是目的寄存器旧值，传到输出。



图 4-2　掩码与结果布局示例。mask=0x55 选中第 0、2、4、6 项，和为 16；低 32 位保存结果，高 96 位清零。

表 4-2　配置和架构状态规定

主题	本实现规定
类型	SEW=8、LMUL=1、VLEN=128；有效向量上下文，vill=0，vtype 保留高位为零
长度	VL=0 保持目的；VL=8～16 固定计算八项；VL=1～7 非法
重启	vstart=0；非零 vstart 触发非法指令异常
掩码及重叠	支持 vm=0/1；允许源/目的重叠及掩码形式 vd=v0，读取旧版本输入
尾部与掩码策略	合法非零运算固定生成完整结果，vta/vma 不改变目的布局
成功状态	保持 vl、vtype、vxrm、vxsat；vstart=0；相关 VS 置 Dirty，包括合法 VL=0
非法状态	进入平台精确异常机制，目的向量及相应向量控制状态保持；陷阱 CSR 按异常路径更新

合法性检查排在 VL=0 保持分支之前，空长度同样要满足权限、类型和重启条件。指令要求 vstart 为零：它不保存中途的部分和，若从中间某个元素重新开始累加，得不到与完整执行相同的结果。完整规格及约定来源见指令规格基线和机器可读参数〔2〕〔3〕。

4.3 接入 VFEX0 的两拍 VDot 单元

算术单元部署在 VFEX0，译码与执行配置中新增指令注册，ROB/CSR 增加异常指令字传递。重命名、发射、唤醒、写回和按序退休沿用既有向量后端。

一条 vdot.vv 译成一个微操作。后端为它准备两个向量源、目的旧值 oldVd、掩码和 VL/类型信息，等这些输入就绪后，将指令发给 VFEX0 中的 VDot 单元。计算结果先写入物理寄存器；等所有更早的指令满足退休条件，ROB 再按序提交这条指令。

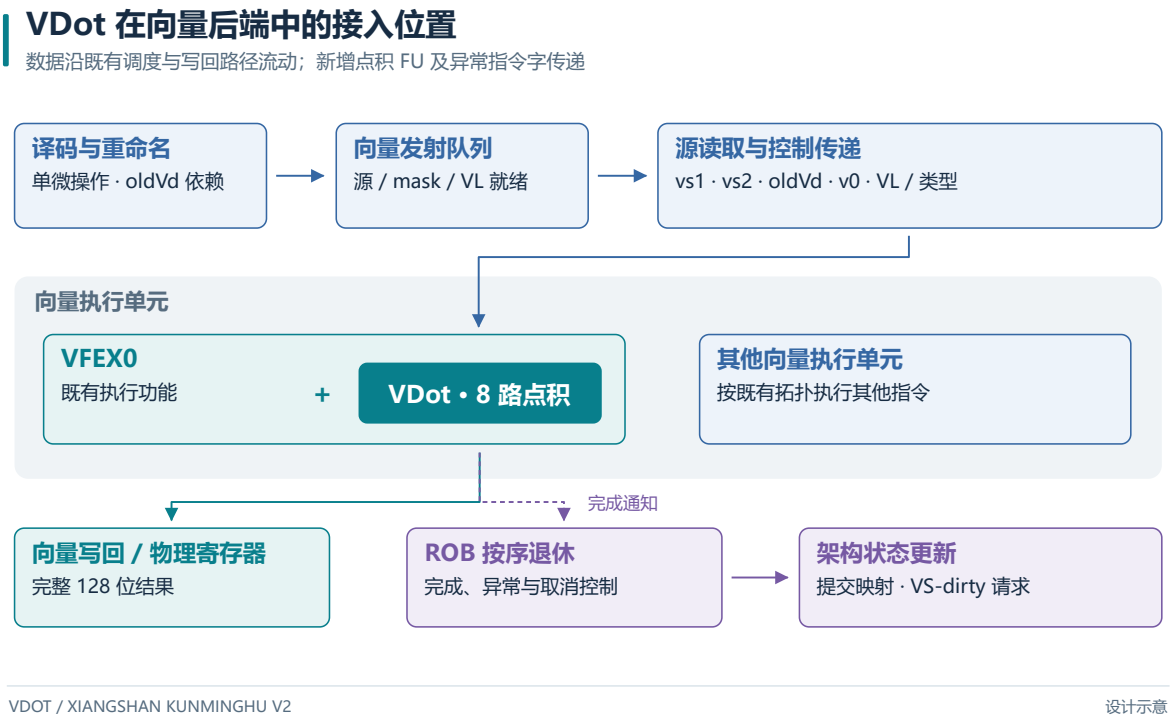


图 4-3　VDot 接入位置。新增部分是专用点积功能单元，寄存器版本管理、依赖检查和退休仍由既有后端承担。

一个微操作完成全部计算，可减少标准组合中的配置切换和分步运算。接入时还需声明 oldVd 依赖、掩码就绪条件及异常信息，使后端能够按完整指令语义调度和提交。第 6、7 章分别给出这些接口的功能与时序检查。

4.4 八路并行乘法与 8→4→2→1 归约树

4.4.1 树形归约与流水切分

八对输入同时进入八个乘法器。八个乘积先两两相加，得到四个和；四个和再两两相加，得到两个和；最后相加得到总和。这就是 8→4→2→1 的树形归约。相较于把八项依次累加，任意一路数据只需经过三级加法器。硬件需要八路乘法器和相应流水寄存器。

第一拍提取字节、完成有符号乘法，并把掩码不选中的乘积置零；乘积存入 S1。第二拍把八个乘积相加，整理成 128 位结果，存入 S2。这样，乘法器和加法树之间隔着一级寄存器。第 6 章用波形检查两拍时序；综合频率和面积尚未测量。

4.4.2 19 位精确和的位宽推导

signed INT8 的范围为 −128…127。单项乘积最小为 −16256，最大为 16384，16 位有符号数足够表示。八项最大和为 `8×16384=131072`，而 18 位有符号最大值是 131071，所以最终归约必须保留第 19 位。

表 4-3　逐级扩位与保存位置

级	并行数量	位宽变化	数据含义
乘法及掩码	8	8×8 → 16 位	每项精确乘积，屏蔽项为零，进入 S1
配对相加	4	16+16 → 17 位	两项部分和
四项相加	2	17+17 → 18 位	四项部分和
八项相加	1	18+18 → 19 位	完整精确和
格式化	1	19 → 32 → 128 位	符号扩展至 32 位，高 96 位补零，进入 S2

负边界 −130048 的低 32 位为 `0xfffe0400`，正边界 131072 为 `0x00020000`。两者的高 96 位均为零。

4.5 两拍数据流与计算示例

掩码选择位于 S1 之前，有掩码与无掩码共用后续归约通路。

输入接受后，分别提取两个源的低八个字节并进行有符号乘法。vm=1 时保留全部乘积；vm=0 时，未使能项替换为 16 位有符号零。S1 保存八个筛选后的乘积。

第二级从 S1 取出八个乘积，先加成四项，再加成两项，最后得到一个 19 位总和。总和符号扩展到 32 位，再在高位补 96 个零，保存到 S2。目的旧值 oldVd 经过另一条流水路径，与点积结果、有效位和 ROB 编号同时到达末级：VL=0 时输出 oldVd，合法非零 VL 时输出点积。

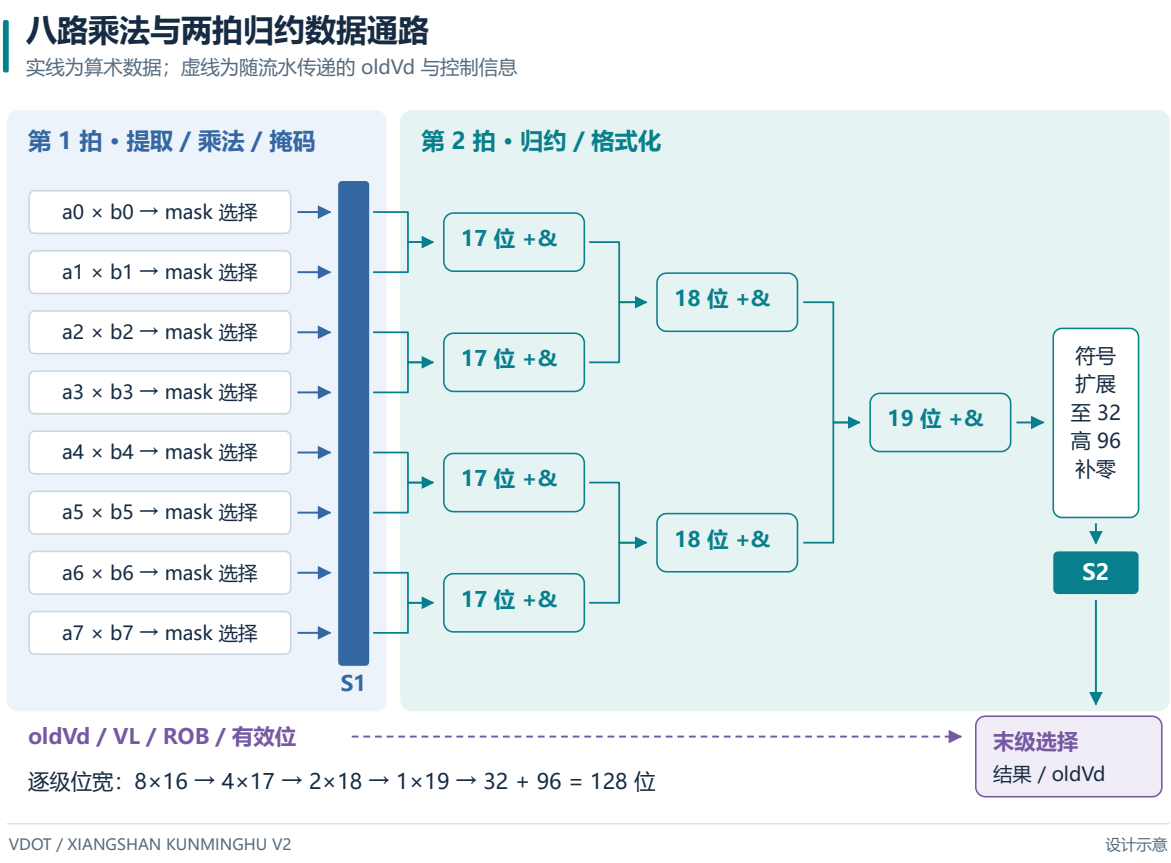


图 4-4　两拍点积数据通路。S1 保存筛选后的乘积，S2 保存格式化结果；oldVd 用于合法 VL=0 的保持。

以 A=[1,2,3,4,5,6,7,8]、B=[1,1,1,1,1,1,1,1] 为例：无掩码时得到 36；掩码 0x55 使第 0、2、4、6 项参与，得到 16。

表 4-4　同一输入的逐级计算

位置	vm=1	vm=0，mask=0x55
乘法后	[1,2,3,4,5,6,7,8]	[1,2,3,4,5,6,7,8]
S1	[1,2,3,4,5,6,7,8]	[1,0,3,0,5,0,7,0]
17 位配对和	[3,7,11,15]	[1,3,5,7]
18 位四项和	[10,26]	[4,12]
19 位总和	36	16
S2 低 32 位	0x00000024	0x00000010
S2 高 96 位	0	0

4.6 VL=0 保持与精确状态更新

4.6.1 VL=0 用 oldVd 把旧值复制到新物理寄存器

假设 v3 原来对应物理寄存器 P10。处理器接收一条写 v3 的 VDot 时，重命名先为它分配新寄存器 P51。若 VL=0，要让程序看到的 v3 不变，就必须把 P10 的全部 128 位复制到 P51。仅仅关闭 P51 的写使能，会让 P51 留下原有内容，无法保证与 v3 的旧值相同。因此，VDot 把 oldVd 也作为输入读取。

译码声明 oldVd 依赖后，执行单元等待旧值就绪，并在合法 VL=0 时将其送出。非零点积不使用 oldVd 进行累加。按本项目状态规定，合法 VL=0 完成后仍将 VS 置 Dirty。

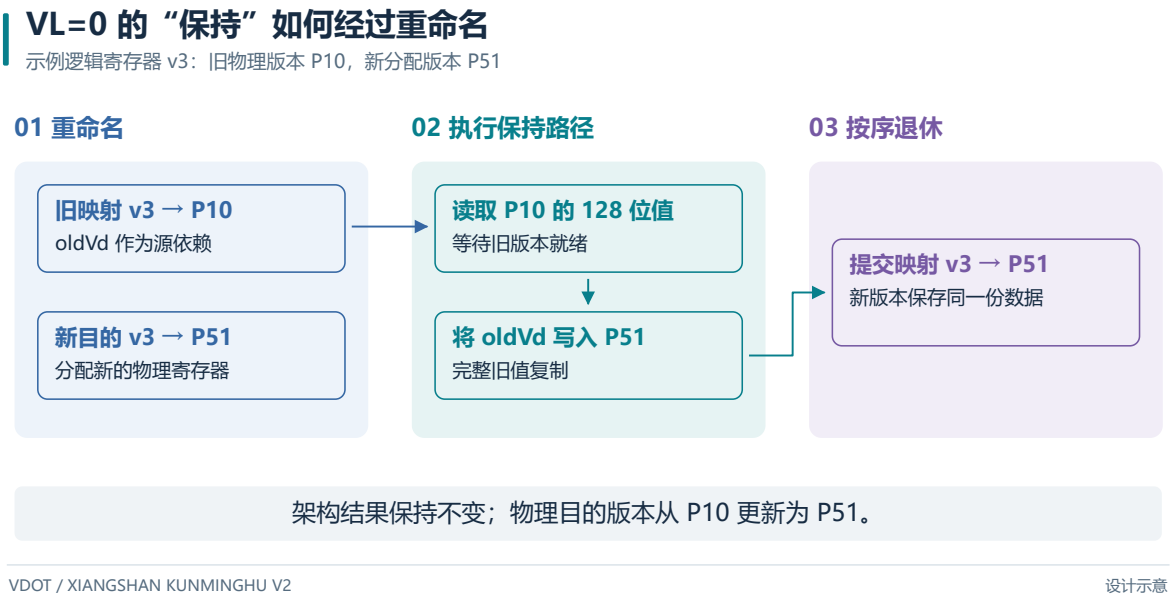


图 4-5　合法 VL=0 的旧值保持。P10、P51 为示例物理编号；旧值复制到新目的后，退休更新逻辑 v3 的已提交映射。

4.6.2 结果只在按序退休后成为架构状态

指令在流水中前进时，数据同时携带 ROB 编号、物理目的和控制信息，后端据此识别结果属于哪一次执行。若较早的分支发现预测路径错误，会发出 redirect，通知后端取消更晚、走错路径的指令。VDot 尚未输出时可直接取消输出；若已经写入物理寄存器，后端恢复仍会阻止它退休。非法配置则通过异常位交给 ROB，等指令到达队首后处理。

验证分别检查算术数值、FU 握手时序及 ROB/CSR 的取消和状态更新。第 5 章说明具体接口，第 6 章给出对应输入与动态波形。

4.7 扩展到更高 SEW / VLEN

当前实现为 SEW=8、VLEN=128、8×INT8→INT32。更高 SEW 和 VLEN 的支持涉及数据宽度、结果语义及后端接口调整，本节给出参数化分析，作为后续设计方向。增大 SEW 会增加乘法器和加法器位宽；固定八项时，归约树仍为三级，掩码仍按每项一位组织。

现有参数约束。 VDot 展开时通过 `require(VLEN == 128 && cfg.destDataBits == 128)` 和 `require(latency == 2 && latdiff == 0)` 固定接口。合法配置为 SEW=8、LMUL=1，低 64 位提供八个输入，19 位精确和写入低 32 位，高 96 位清零。

扩大元素位宽。 设每个结果归约 n 项，各部件的位宽关系见表 4-5。八项 INT16 输入占满 128 位源，乘积为 32 位，最大总和为 $8\times2^{30}=2^{33}$，需要 35 位有符号数，可使用 INT64 容器保存。合法 VL 范围相应按 VLMAX=VLEN/SEW 重新规定。

表 4-5　参数化后各部件的变化（设计分析，无实测）

部件	一般形式	当前：SEW=8，n=8	示例：SEW=16，n=8
乘法器位宽	SEW×SEW → 2·SEW 位	8×8 → 16 位	16×16 → 32 位
归约树深度	平衡二叉树至多 $\lceil\log_2 n\rceil$ 级	3 级	3 级
精确和位宽	采用 $2\cdot SEW+\lceil\log_2 n\rceil$ 位可安全容纳	19 位	35 位
结果放置	结果容器宽度由扩展规格约定，须容纳精确和且不超过目的宽度	INT32 写 [31:0]，高 96 位清零	可选 INT64 写 [63:0]，高 64 位清零
源读取范围	n·SEW 位	低 64 位	完整 128 位
掩码粒度	每项一位，屏蔽项替换为 2·SEW 位零	old_v0 低 8 位，16 位零	old_v0 低 8 位，32 位零

后端接口。 两源、oldVd、掩码和 VL 输入，以及配置与掩码唤醒仍需保留。更宽数据通路能否继续采用一个微操作、一次目的更新和两拍唤醒，要结合寄存器堆分片、写回端口及时序重新确定。VL=0 保持完整旧目的，非法配置进入精确异常通路。编码字段不变时可沿用 mtval 重建方法；另设编码时，识别和重建逻辑需同步修改。

按段扩展 VLEN。 以 SEW=8、VLEN=256 为例，可设置两份八路单元。段 k 读取本段低八项，将结果放入本段低 32 位并清零本段高 96 位；第 i 项对应 v0 的第 16k+i 位。该方案产生两个段结果，需要定义新的多结果语义，并明确各段与 VL 的关系、未激活段的处理及编码区分，同时更新 RTL、NEMU 和测试。结构上还需考虑以下问题：

- 每段仍为三级加法树，算术层级不因段数增加；更宽的总线、扇出和布线仍可能影响时序，两拍和 `CertainLatency(2)` 需重新验证。若改为跨全部元素归约，项数每翻倍才增加一级加法，流水切分须结合时序结果选择。
- 每段算术可沿用八项求和与结果格式化的做法；第 6 章的检查器和第 7 章的参考模型需增加多段输出、VL 和掩码测试，当前通过记录仍只覆盖 VLEN=128。
- 需要跨段总和时，软件可再用一次标准归约合并各段结果。这部分开销需另行测量，本文不作估计。

5 RTL 实现与整核接入

接入 VDot，需要让处理器能认出指令、读到正确输入、送到新的计算单元，并把结果或异常交回后端。本章沿这条顺序说明 DecodeUnit、FuConfig、VDot 和 ROB/CSR 的修改。指令编码、向量分类、FuType、执行拓扑和异常 Bundle 的完整修改保存在交付源码与补丁中。

算术单元收到输入后能算出正确答案，还需要后端配合：输入未就绪时等待，完成时唤醒使用结果的指令，遇到异常或分支取消时停止提交。执行类型、源依赖、固定延迟和 ROB 编号就是这些配合所需的信息。

源码以锁定的香山基版本叠加 VDot 及 mtval 修复补丁为准。下文为保存源码中的关键摘录，完整文件与补丁见阶段二复现索引。

5.1 DecodeUnit：单微操作与 oldVd 依赖

custom-0 位型被识别后，指令仍进入向量分类和权限检查通路。DecodeUnit 的设置如下：

```
when(inst.isVdot) {
  io.deq.isComplex := true.B
  io.deq.uopInfo.numOfUop := 1.U
  io.deq.uopInfo.numOfWB := 1.U
  io.deq.uopInfo.lmul := 1.U
}
```

`isComplex` 使 VDot 进入对应的向量译码路径；`numOfUop=1` 规定八项点积只生成一个微操作，`numOfWB=1` 规定一次目的更新。`uopInfo.lmul` 是内部字段，不能直接当作软件 vtype 的编码；软件设置 LMUL=1 时，`vtype.vlmul` 的编码为 0。

`isDependOldVd` 加入 `inst.isVdot` 后，调度端知道 VDot 还要读取目的旧值。旧值未就绪时，指令会等待；旧值就绪后，合法 VL=0 可以把它复制到新物理目的。非零 VL 的点积仍从零开始求和〔4〕。

5.2 FuConfig：把 VDot 挂入 VFEX0

`FuType.vdot` 标识新增执行类型，执行拓扑将对应 `VdotCfg` 配置挂入 VFEX0。配置列明输入类型、两拍固定延迟和结果写回能力：

```
  val VdotCfg = FuConfig (
    name = "vdot",
    fuType = FuType.vdot,
    fuGen = (p: Parameters, cfg: FuConfig) => Module(new VDot(cfg)(p)),
    srcData = Seq(Seq(VecData(), VecData(), VecData(), V0Data(), VlData())),
    piped = true,
    writeVecRf = true,
    writeV0Rf = true,
    latency = CertainLatency(2),
    vconfigWakeUp = true,
    maskWakeUp = true,
    destDataBits = 128,
    exceptionOut = Seq(illegalInstr),
  )
```

前三个 `VecData()` 分别提供两个向量源和 oldVd，`V0Data()` 提供掩码，`VlData()` 提供长度。`maskWakeUp` 和 `vconfigWakeUp` 让后端等正确的掩码与配置就绪后再发射。`writeV0Rf` 支持把 v0 作为目的，`destDataBits=128` 规定写回完整向量。`CertainLatency(2)` 则告诉后端：功能单元按两拍产生结果，唤醒与写回安排需要与之匹配。

这份配置决定算术模块的输入来源和结果写回方式。完整定义见 FuConfig 源码〔5〕。

5.3 VDot 模块：有符号乘法、+& 归约与 S1/S2 寄存器

VDot 继承 `VecPipedFuncUnit`，使用父类提供的流水控制。模块展开时先检查向量宽度、结果宽度及两拍延迟约束：

```
class VDot(cfg: FuConfig)(implicit p: Parameters) extends VecPipedFuncUnit(cfg) {
  require(VLEN == 128 && cfg.destDataBits == 128)
  require(latency == 2 && latdiff == 0)
```

算术主体如下：

```
  val products = VecInit((0 until 8).map { i =>
    val product = vs1(8*i+7, 8*i).asSInt * vs2(8*i+7, 8*i).asSInt
    Mux(srcMask(i), product, 0.S(16.W))
  })
  val p1 = S1Reg(products)
  val pairs = (0 until 4).map(i => p1(2*i) +& p1(2*i+1))
  val halves = (0 until 2).map(i => pairs(2*i) +& pairs(2*i+1))
  val sum = halves(0) +& halves(1) // 19 bits, including +131072
  val result = S2Reg(Cat(0.U(96.W), sum.pad(32).asUInt))
```

`.asSInt` 把取出的字节当作有符号数，例如 `0x80` 按 −128 参与乘法。掩码未选中的乘积在进入 S1 前替换成 16 位零。`+&` 在相加后多保留一位，所以三级结果依次为 17、18、19 位。`sum.pad(32)` 将符号位扩展到 32 位，`Cat` 再在高位拼接 96 个零。

`S1Reg` 保存第一拍的八个乘积，`S2Reg` 保存第二拍整理好的结果。有效位、ROB 编号和控制信息与数据一起推进，因此末级既知道算出了什么，也知道结果应交给哪条指令〔6〕。

5.4 末级：结果选择、非法检查与 redirect 取消

末级先检查类型、vstart 和 VL。合法非零 VL 输出 S2 点积结果；VL=0 输出 oldVd；局部检查发现非法配置时，也选择 oldVd，并同时置起 `illegalInstr`。ROB 根据异常位进入精确异常处理。输出是否有效，还要看该指令有没有被 redirect 取消。

```
val illegal = outVecCtrl.vill || outVecCtrl.vsew =/= 0.U ||
  outVecCtrl.vlmul =/= 0.U || outVstart =/= 0.U ||
  (outVl =/= 0.U && (outVl < 8.U || outVl > 16.U))
io.out.bits.res.data := Mux(outVl === 0.U || illegal, outOldVd, result)
io.out.bits.ctrl.exceptionVec.get(illegalInstr) := illegal
// Keep vecWen/v0Wen for legal vl=0: the new physical destination needs oldVd.
// CSR updates and precise exceptions remain on the ordinary commit path.
io.out.valid := fixValidVec.last && !outCtrl.robIdx.needFlush(io.flush)
```

向量权限和部分类型检查位于上游，这里保留的是 FU 局部保护。

`io.out.valid` 要求末级有有效指令，且 `needFlush` 没有判定它需要取消。ROB 指针的 value 表示槽位，flag 用于区分队列绕回后的代际；比较完整指针，才能判断 VDot 是否位于发出 redirect 的较早分支之后。若 redirect 到来前已经写回，ROB 和重命名恢复会阻止该结果提交。因此，波形分别记录“写进物理寄存器”和“按序退休”两个时刻。

向量脏状态（VS=Dirty）请求由同一拍各退休槽的标记汇总得到：

```
VS-dirty 请求来源 = isCommit AND OR(commitValid[i] AND dirtyVs[i])
```

只有 `isCommit` 和对应槽的 `commitValid` 同时有效，该槽的 `dirtyVs` 才计入请求。槽里即使残留 `dirtyVs=1`，只要 `commitValid=0` 就不参与。第 6 章分别用正常退休和分支取消的波形核对这一过程。

5.5 mtval 修复：ROB 保存编码字段，队首重建指令字

VL=1～7 等非法条件要到执行期才能判定。异常发生后，CSR 的 mtval 需要保存故障指令的机器码。为使指令到达 ROB 队首时仍能取到该编码，新增表格在 ROB 分配时记下 VDot 的可变字段，到队首异常时再拼回完整指令字。

协同仿真曾在 short-1-vm0 用例中发现 mtval 失配：参考值为 `0x0020818b`，DUT 为零。原 trapInstInfo 主要服务于译码期非法异常，执行期触发的 VDot 到达队首时缺少相应指令字。完整诊断过程见第 7 章。

新增编码表按 ROB 槽保存类型、代际和可变字段，核心实现如下：

```
val isVdot = RegInit(VecInit(Seq.fill(RobSize)(false.B)))
val generation = Reg(Vec(RobSize, Bool()))
val fields = Reg(Vec(RobSize, UInt(16.W)))
for (a <- io.alloc) {
  when(a.valid) {
    val idx = a.bits.robIdx.value
    isVdot(idx) := VdotInstructions.VDOT_VV === a.bits.instr
    generation(idx) := a.bits.robIdx.flag
    fields(idx) := Cat(a.bits.instr(25), a.bits.instr(24,20),
                       a.bits.instr(19,15), a.bits.instr(11,7))
  }
}
val f = fields(io.head.value)
val word = Cat(0.U(6.W), f(15), f(14,10), f(9,5), 0.U(3.W), f(4,0), "h0b".U(7.W))
val matched = isVdot(io.head.value) && generation(io.head.value) === io.head.flag
// Same one-cycle capture as Rob.io.exception; no writeback-order arbitration.
io.out.valid := RegNext(io.trap && io.illegal && !io.interrupt && matched, false.B)
io.out.bits := RegEnable(word, io.trap)
```

`vm`、`vs2`、`vs1`、`vd` 共 16 个可变位，加一个类型位和一个代际位，每槽 18 位；48 槽共 864 个存储字段位。固定的 funct6、funct3 和 opcode 可在异常时重建，无需逐槽保存完整的 32 位指令字。以上为字段容量估算，综合面积另需实测。

队首发生非法指令异常时，`matched` 确认类型和代际，`!interrupt` 排除中断，匹配信息随后与 ROB 异常信息同步寄存。CSR 用选择器优先接收该信息，其他异常保留原有通路：

```
csrMod.io.trapInst := xiangshan.backend.rob.VdotTrapInstSelect(
  csrIn.exception.valid, csrIn.exception.bits.isInterrupt,
  csrIn.exception.bits.exceptionVec(EX_II), csrIn.exception.bits.vdotTrapInst,
  trapInstMod.io.currentTrapInst)
```

指令分配 ROB 槽位时先保存编码字段；到队首发生非法异常时，核对类型和代际；匹配后重建 32 位指令字，再交给 CSR 写入 mtval。取值依据是发生异常的 ROB 实例，因此不受各功能单元谁先算完的影响〔7〕〔8〕。



图 5-1　mtval 修复的数据路径。ROB 分配时保存可变编码字段与代际，队首异常时重建指令字，并与异常信息同步送至 CSR。

修复版 emu 使用原异常用例回归，64 例 EXCEPTION 全部通过，RTL 与参考模型的 mtval 一致。故障 PC、失败快照和回归记录见第 7 章。

5.6 补丁、emu、参考库与测试镜像的对应关系

复现时分别构建 RTL 仿真器、NEMU 参考库及 RISC-V 测试镜像，再按保存的版本与配置组合运行。三者的源码、补丁和文件校验值见阶段二附录。

完整交付补丁已包含 VDot 接入与 mtval 修复。后续功能、等价和性能实验复用锁定的修复版 emu 与源码 NEMU；仅调整测试输入或程序时，重新编译应用即可〔9〕。

6 功能测试与补充时序验证

vdot-smoke 用七组输入检查 VDot：普通正数、正负极值、按位掩码、零掩码、VL=0 保持，以及 VL=16 时只算低八项。每组都比较完整 128 位目的和规定的状态，已有整核记录全部通过。随后用波形查看 VDot 两拍完成计算、按序退休，以及先算完但被较早分支取消的过程。

测试先选能手算的输入，由 C 程序独立算出答案，再与硬件保存的全部 16 字节结果比较。波形检查继续跟踪同一次执行：什么时候收到输入、什么时候输出、是否遇到分支取消、最终有没有退休。

正数、负数和边界测试分别对应赛题 2.3.1～2.3.3，后半部分给出补充时序验证。异常、寄存器重叠及流水套件的差分结果在第 7 章汇总。

6.1 vdot-smoke：C 检查器逐字节判定 128 位目的

测试程序由 C 检查器和汇编包装组成。C 程序构造输入并独立计算期望值，汇编负责装载源、掩码和旧目的，执行自定义机器码，随后捕获状态并存回完整结果。测试在修复版单核 MinimalConfig 上运行，使用源码 NEMU 与固定 Flash，同时启用应用自检和 DiffTest。

输入、旧目的和输出数组都按 16 字节对齐。测试先给旧目的写入 `0xa0…0xaf`，给输出缓冲区填入 `0xee`，并将 A、B 的高八字节分别填为 127、−128。这样，目的未写回、VL=0 未保留旧值、高半输入被误算，都能在比较中暴露出来。

汇编先用 e8/m1、VL=16 装入完整寄存器，再切换到当前用例的 VL。执行 VDot 前，把 VS 设回 Initial，并设置 vxrm=3、vxsat=1、vstart=0。VDot 执行后立即读取 VS 和 VL，再重新配置并存储 v3。先读状态、后存结果，是为了让 VS 检查对应 VDot 本身，避免把后续向量操作设置的 Dirty 算进去。

C 检查器的核心判定如下，直接覆盖全部 16 字节：

```
for (unsigned i = 0; i < 16; ++i) {
  uint8_t expected = vl == 0 ? old[i] : i < 4 ? (uint32_t)sum >> (8*i) : 0;
  if (out[i] != expected) {
    printf("FAIL vdot case=%d byte=%d got=%x expected=%x\n",c,i,out[i],expected); return 1;
  }
}
```

非零 VL 下，低四字节保存小端 32 位点积，其余十二字节必须为零；VL=0 则逐字节与 old 比较。程序还要求 VL 与请求一致、VS=Dirty、vcsr=7、vstart=0。任一检查失败立即返回 1，全部用例通过才输出 PASS。实际执行顺序见完整 C 源码与汇编包装〔10〕〔11〕。

6.2 正数点积 case0 / case3：得到 36 与 16

无掩码用例 case0 取 A=[1,2,3,4,5,6,7,8]、B=[1,1,1,1,1,1,1,1]，VL=8、vm=1。八项乘积分别为 1…8，总和为 36，完整目的应为：

```
vd = 0x00000000_00000000_00000000_00000024
```

case0 同时检查八项求和和高 96 位清零。旧目的预填非零值，遗漏高位清零会直接触发完整目的比较失败。

case3 使用相同输入，改为 vm=0、mask=0x55。从最低位起只有 i=0、2、4、6 使能，期望变为 1+3+5+7=16，完整结果为 0x00000000_00000000_00000000_00000010。这个用例能区分按位掩码与错误的逐字节解释，并确认掩码作用于乘积本身。

表 6-1　正数测试的期望与记录

用例	输入与使能	期望点积	实测判定
case0	VL=8，vm=1，全部八项	36	sum=36 full128=OK VS=Dirty
case3	VL=8，vm=0，mask=0x55	16	sum=16 full128=OK VS=Dirty

6.3 负数点积 case2：−130048 只在低 32 位带符号

负数用例 case2 的实测记录为 sum=-130048 full128=OK VS=Dirty。该用例重点检查有符号乘法、精确归约，以及负数在低 32 位中的表示。

case2 的前八对输入均为 −128 与 127，单项乘积为 −16256，八项和为 −130048。若把 0x80 当作无符号 128，结果符号会出错；若归约位宽不足，也得不到精确结果。

19 位和符号扩展到低 32 位后为 0xfffe0400，再与高 96 位零拼接：

```
S  = 8 × (−128) × 127 = −130048
vd = 0x00000000_00000000_00000000_fffe0400
低地址起的字节 = 00 04 fe ff 00 00 00 00 00 00 00 00 00 00 00 00
```

检查器对全部 16 字节逐一比较，既核对低 32 位补码，也核对高 96 位零。该输入同时达到本运算的负极值，可检查错误的无符号计算、归约截断或向整个目的进行符号扩展等问题。

6.4 边界用例 case1/4/5/6：19 位和、零掩码、VL=0 与 VL=16

边界测试分别针对归约上界、零掩码、空长度保持和高半输入隔离。连同前两节的正负数测试，七个用例均通过完整目的与指定状态检查，整核运行给出 DiffTest、GOOD TRAP 及退出码 0。

6.4.1 case1：131072 检验第 19 位

case1 取八对 −128×−128，得到 `8×16384=131072`，低 32 位为 `0x00020000`。该值恰比 18 位有符号上界多 1，直接检验归约树是否保留了第 19 位。正、负极值共同覆盖了精确和的两端。

6.4.2 case4 / case5：零掩码清零，VL=0 保持

case4 为 VL=8、vm=0、mask=0，全部乘积被屏蔽，归约结果及整个目的应为零。

case5 设置合法 VL=0，预期目的仍为原来的 a0 a1 … af。按高位在左显示，完整值是 `0xafaeadac_abaaa9a8_a7a6a5a4_a3a2a1a0`。检查器逐字节比较新结果和 old。日志里的 `sum=36` 是 C 程序提前算好的点积字段；VL=0 分支实际检查的是旧目的保持，成功标志看 `full128=OK`。

6.4.3 case6：VL=16 仍只算低八项

case6 把 VL 设为 16，低八项与 case0 相同，高八对特意填入 127×−128。按固定八项规格，结果仍为 36；若误算高半部，会额外加入 −130048。该输入能识别“按 VL 把全部十六项归约”的错误实现。

表 6-2　边界用例及其完整目的要求

用例	检查对象	完整目的要求	实际结果
case1	正极值、19 位和	低 32 位 `0x00020000`，高位零	通过
case2	负极值、符号扩展	低 32 位 `0xfffe0400`，高位零	通过
case4	零掩码	完整 128 位零	通过
case5	VL=0	完整旧目的保持	通过
case6	VL=16、高半部隔离	低 32 位 36，高位零	通过

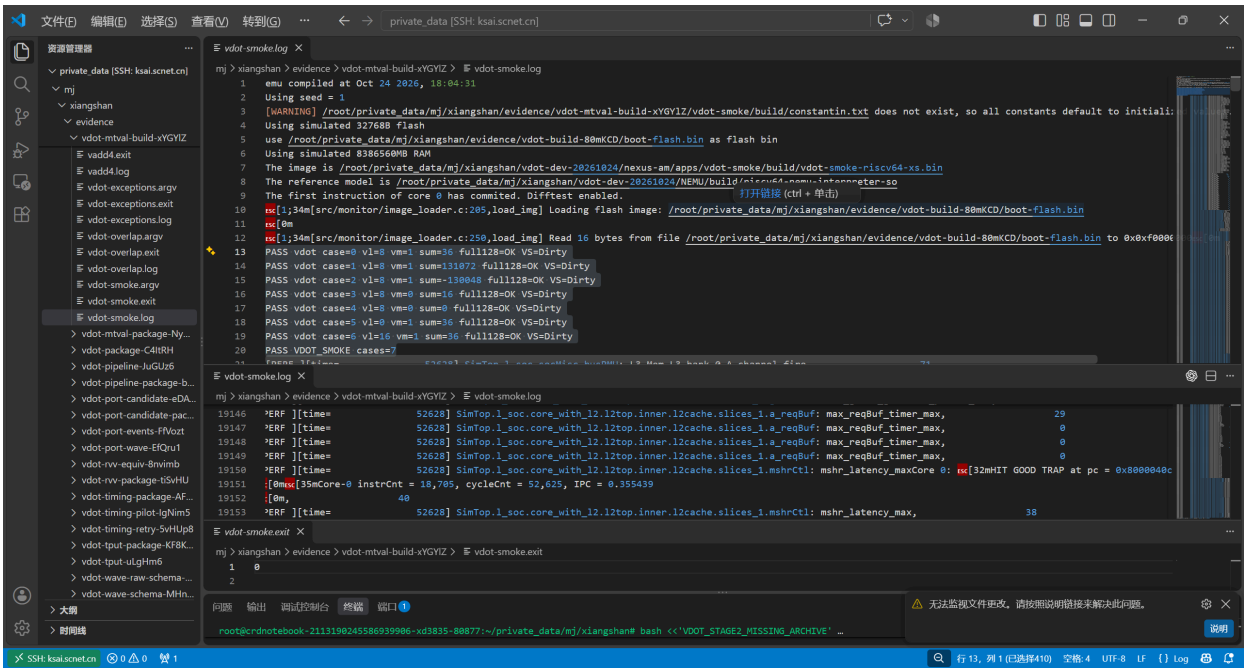


图 6-1　七例基础功能测试。正数、负数、极值、掩码及保持用例各自执行完整目的检查，原始记录见修复版整核回归〔12〕。

6.4.4 在提交仓库运行基础功能测试

在已经编译好的本机环境中，启动 Docker Desktop，在提交仓库根目录的 Windows PowerShell 运行：

```
python tools/container.py exec python3 tools/recording_case.py vdot-smoke
```

命令会调用现有 emu 和 NEMU，运行七个基础用例，并自动汇总本次输出。成功时应看到七条 `PASS vdot case=`、`PASS VDOT_SMOKE cases=7`、`Difftest enabled.`、`HIT GOOD TRAP`，以及 `EMU_EXIT_CODE=0`、`RUNNER_EXIT_CODE=0`、`RUN_CHECK=PASS`。日志目录以本次终端打印的 `Run directory` 为准；环境准备和整组验收见复现指南。

6.5 正常实例：输入到 FU 完成 2 周期，到退休 26 周期

正常实例中，VDot 收到输入后用 2 周期算出结果，又经过后端等待，直到输入后的第 26 周期才在 ROB 退休。记录这两个时刻，可以分别看清计算用了多久、结果又等了多久才提交。

波形窗口的时钟周期为 2 ps，握手按事件沿前的稳定值判定。每个动态实例使用 PC、机器码、完整 ROB flag/value、pdest 和分配区间关联。正文中的 ROB/pdest 统一使用十进制。

正常实例 PC=`0x80000208`，机器码=`0x0220818b`，ROB=1/36，pdest=62。它在 14208 ps 分配，14220 ps 接受 FU 输入，14224 ps 输出完整结果，14272 ps 在 ROB 退休。

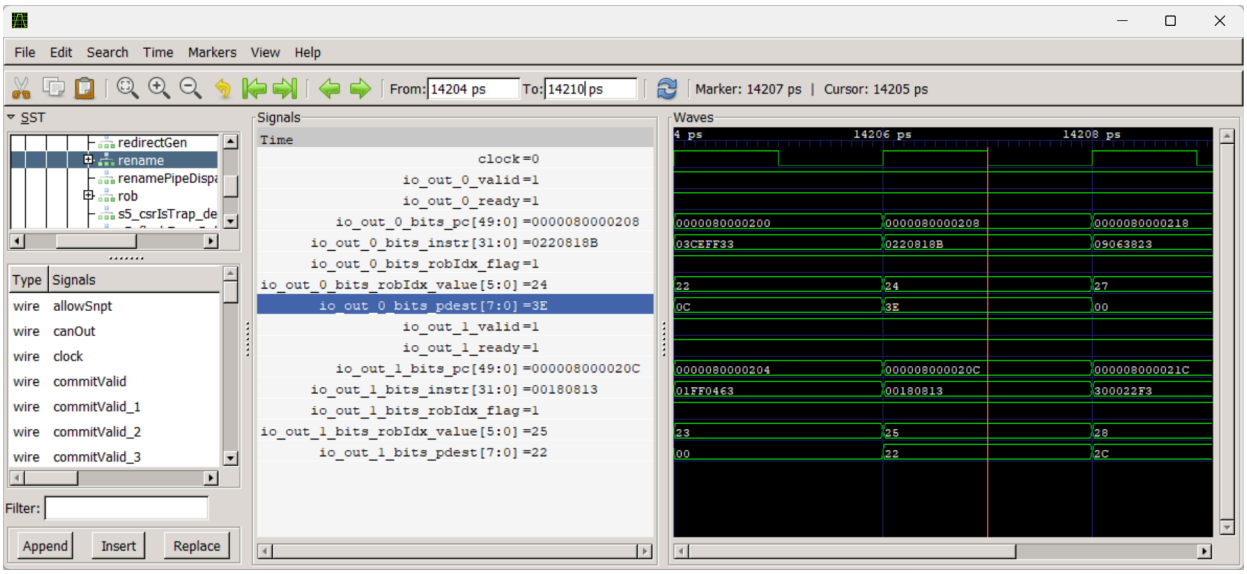


图 6-2　14207 ps 的分配沿前身份。波形中的 ROB 1/0x24、pdest 0x3e 对应正文的 1/36、62。

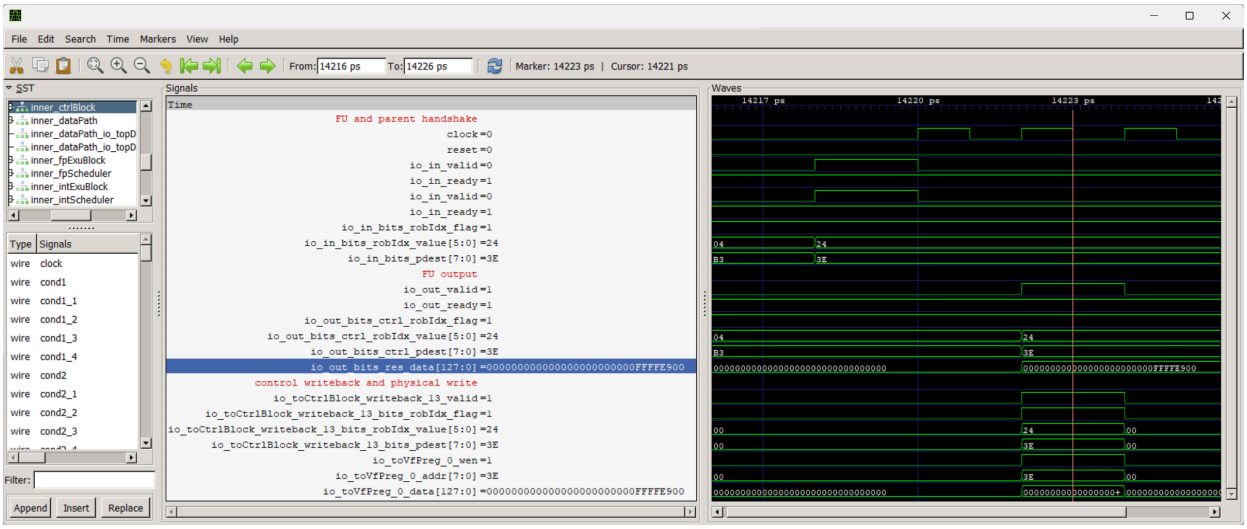


图 6-3　输入与输出相隔 4 ps，即 2 周期；输出为 `0x00000000000000000000000ffffe900`，低 32 位表示 −5888。

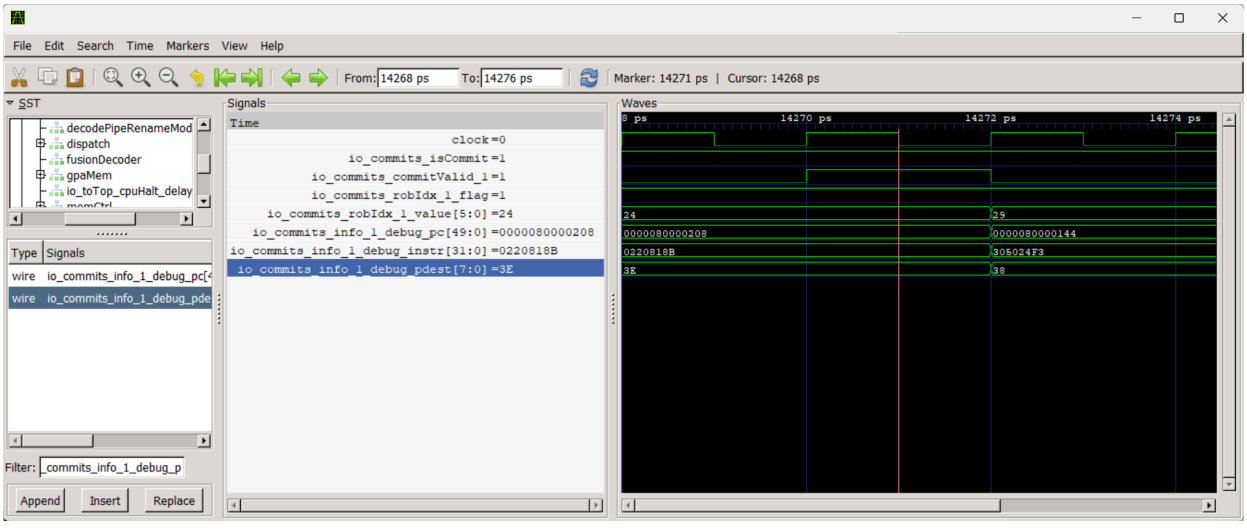


图 6-4　14271 ps 的 lane1 满足有效退休条件，ROB/pdest 与正常实例一致，对应 14272 ps 退休。

上述时间差分别对应功能单元计算和按序提交过程，完整动态事件记录见正常与取消窗口核验〔13〕。

6.6 取消实例：先完成的投机 VDot 未被提交

另一实例中，VDot 于 16908 ps 完成计算，较老分支于 16952 ps 发出 redirect。检查持续到原 ROB 身份在 16970 ps 被复用，期间该 VDot 没有有效退休。

取消实例执行的还是 PC=`0x80000208` 的同一条机器码，但这是程序再次运行到该位置时的一次新执行，分到 ROB=1/27、pdest=61。它前面的分支位于 PC=`0x80000204`，ROB=1/26。处理器先按预测路径执行 VDot，因此可能在分支方向确定前就算出结果。

表 6-3　投机实例的事件顺序

事件	时间	相对 VDot 输入周期
较老分支分配	16890 ps	−7
VDot 分配	16892 ps	−6
VDot 输入	16904 ps	0
VDot 完成	16908 ps	2
分支 redirect，覆盖年轻 VDot	16952 ps	24
分支退休	16958 ps	27
VDot 原 ROB 身份复用	16970 ps	33

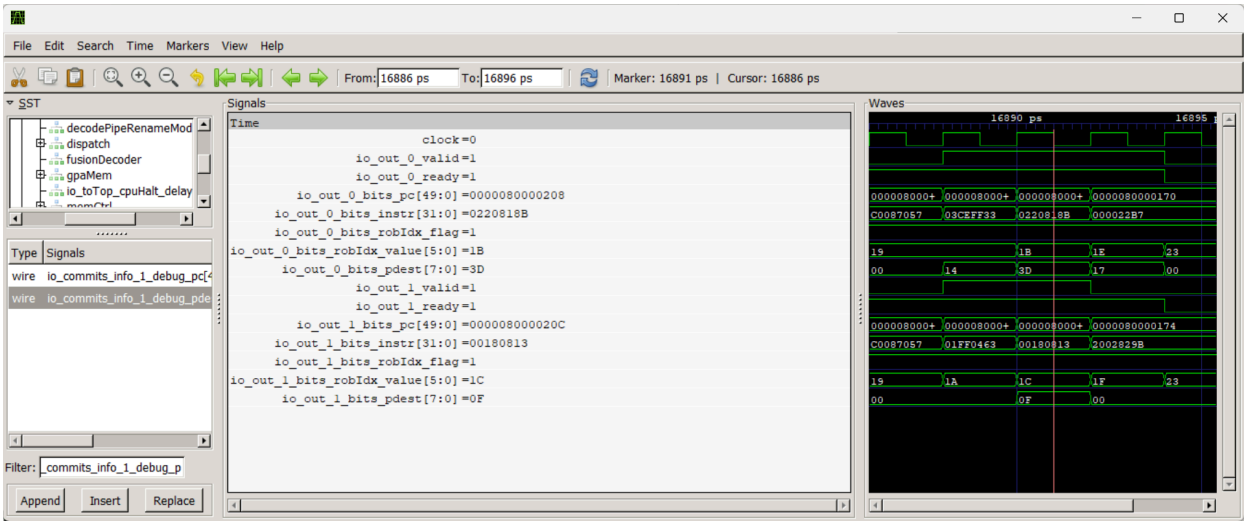


图 6-5　VDot ROB 1/0x1b、pdest 0x3d 在较老分支 ROB 1/0x1a 之后分配。

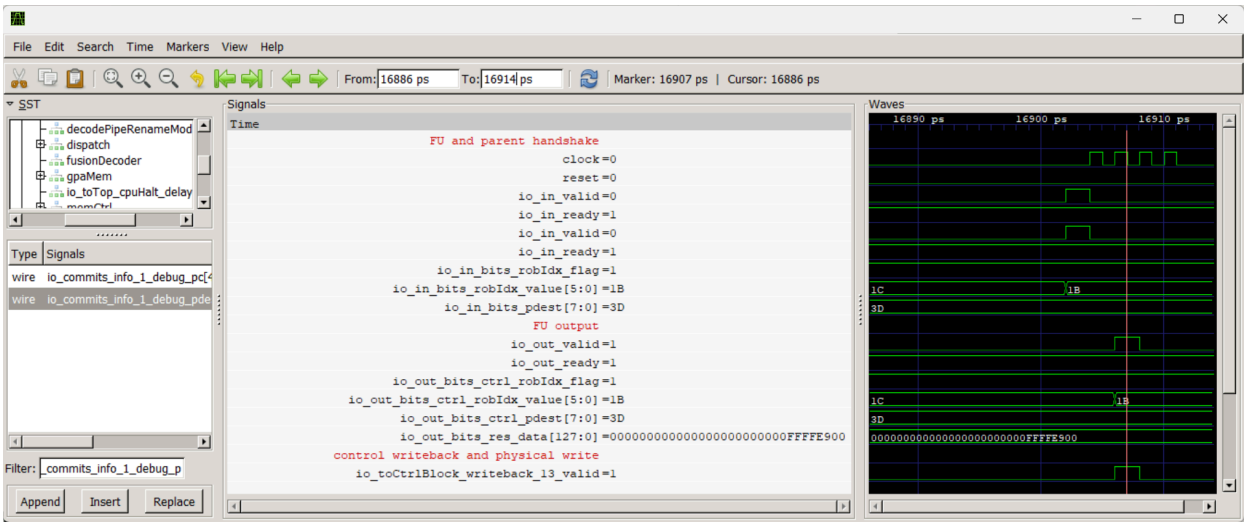


图 6-6　投机 VDot 在 16904、16908 ps 分别输入和完成，输出仍携带本次 ROB/pdest。

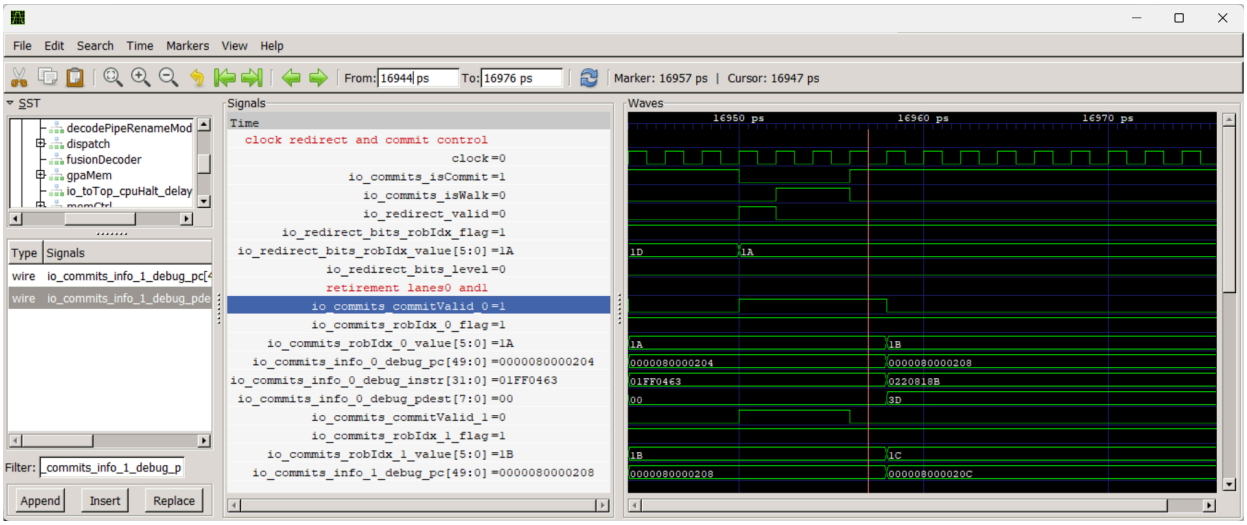


图 6-7　较老分支在 16952 ps 产生 redirect，随后退休；年轻 VDot 对应槽的身份数据没有形成有效退休。

输入和输出握手表明 VDot 确实完成了计算。较早分支随后发现路径需要纠正，发出 redirect；从该时刻到 ROB 身份被复用，记录中都没有这条 VDot 的有效退休。处理器可以提前计算预测路径上的指令，但只有没有被取消、能够按序退休的结果才会成为已提交状态。

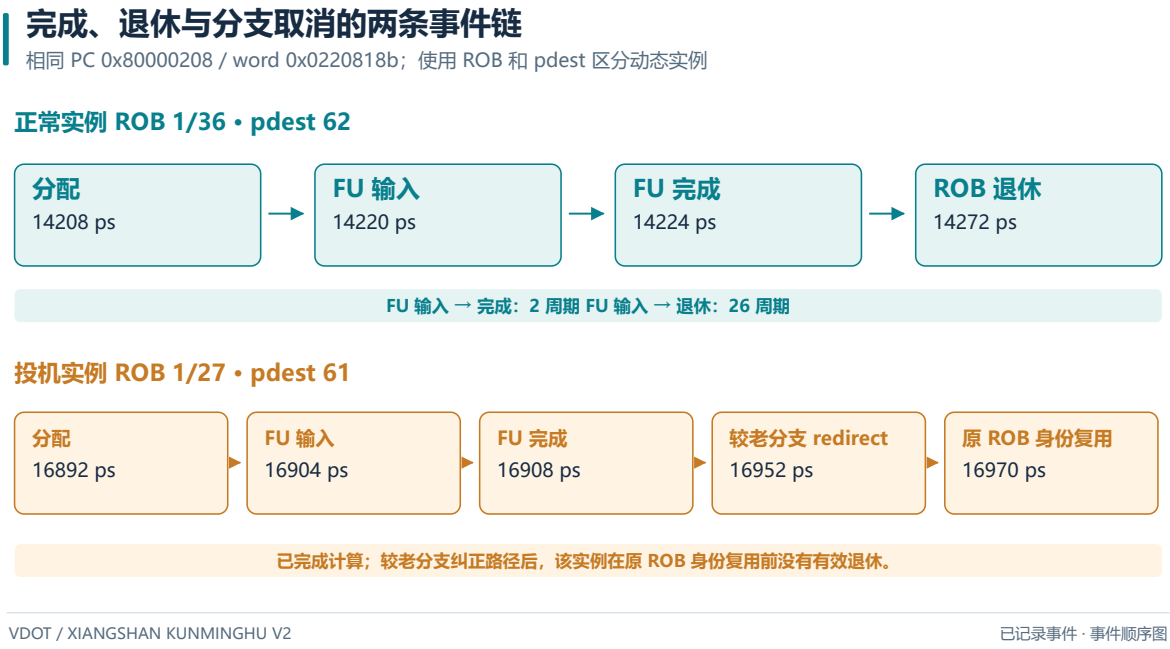


图 6-8　正常与投机实例的事件对照。两者均在 FU 输入后 2 周期完成；正常实例退休，投机实例在较老分支恢复后没有有效退休。各框按事件顺序排列，时间见框内。

6.7 VS-dirty 请求与共享执行资源

本节检查退休产生的 VS-dirty 请求，以及 VDot 与其他向量指令共用执行资源时的行为。状态请求按逐拍记录比较，资源使用按具体动态实例分析。

6.7.1 VS-dirty 只来自有效退休贡献

检查保存窗口中的 2280 个非复位上升沿时，先用各退休槽的 `isCommit AND OR(commitValid[i] AND dirtyVs[i])` 算出应有请求，再与 ROB 寄存后的请求和 CSR 收到的请求逐拍比较。结果没有差异，其中八次正常 VDot 退休都传递了对应请求。

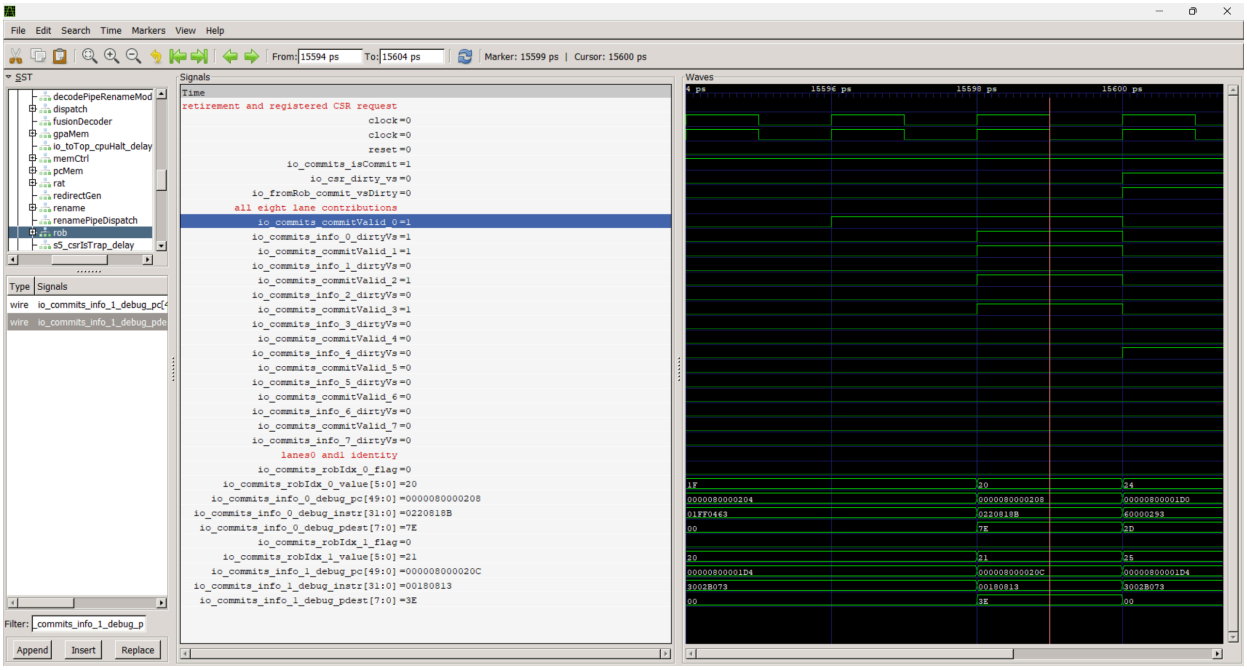


图 6-9　15599 ps 的 VDot 有效退休贡献在 15600 ps 沿后形成 ROB 与 CSR 请求。该例 ROB 为 0/32、pdest=126。

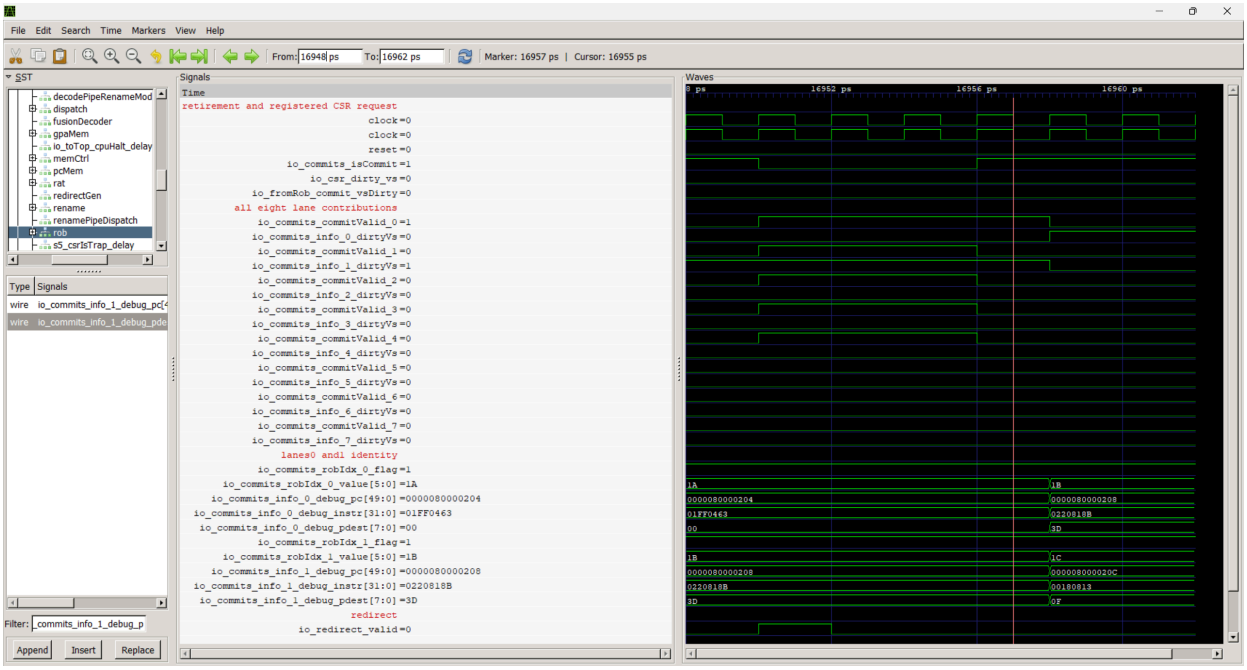


图 6-10　取消例中 VDot 槽虽然有 dirtyVs=1，但 commitValid=0，16958 ps 沿后请求保持为零。

两例说明，只有有效退休的槽才计入汇总，槽内残留的 dirtyVs 位不会单独触发请求。逐拍比较覆盖从退休槽标记到 CSR 接收请求的全过程；VS 寄存器有多个更新来源，它们同时出现时的优先级需另设用例检查。原记录见 CSR 请求核验〔14〕。

6.7.2 混发可重叠推进，端口竞争延后两周期获准

混发测试中，VADD 使用 EXU2，VMUL 和 VDot 使用 EXU0。三条指令的输入、完成和退休顺序如下：

表 6-4　混发实例，时间单位为 ps

指令	ROB / pdest	输入	完成	退休
VADD，PC=0x800001b8	1/35 / 62	13216	13218	13222
VMUL，PC=0x800001bc	1/36 / 63	13218	13222	13226
VDot，PC=0x800001c0	1/37 / 64	13220	13224	13228

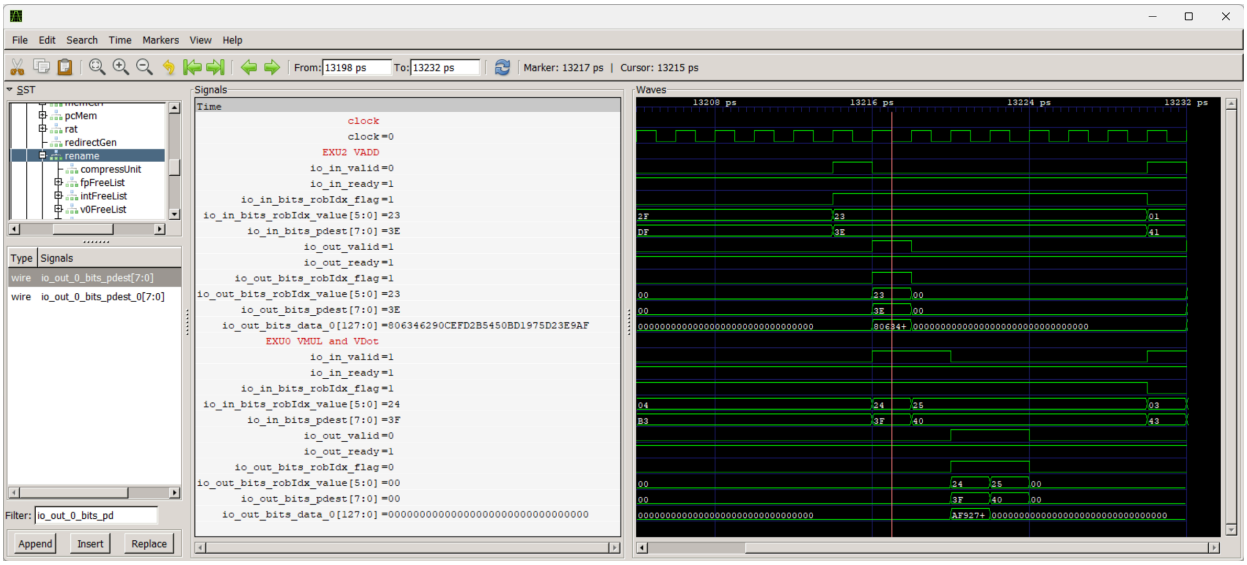


图 6-11　Marker=13217 ps 显示 VADD 输出与 VMUL 输入的有效条件，随后 VMUL、VDot 连续完成。

VMUL 与 VDot 的执行区间发生重叠，三条指令均正常退休。另一个定向程序安排 VDot 与 VIOTA 同时申请发射前 VfWB 端口 0，用于观察资源冲突时的仲裁与后续推进。

15658 ps 时两条请求同时有效：VDot（ROB=0/46，pdest=56）获准，VIOTA（ROB=0/47，pdest=57）的 ready 为零。VIOTA 于 15662 ps 再次申请并获准。两条指令随后分别在 15668、15672 ps 完成，在 15672、15676 ps 退休。

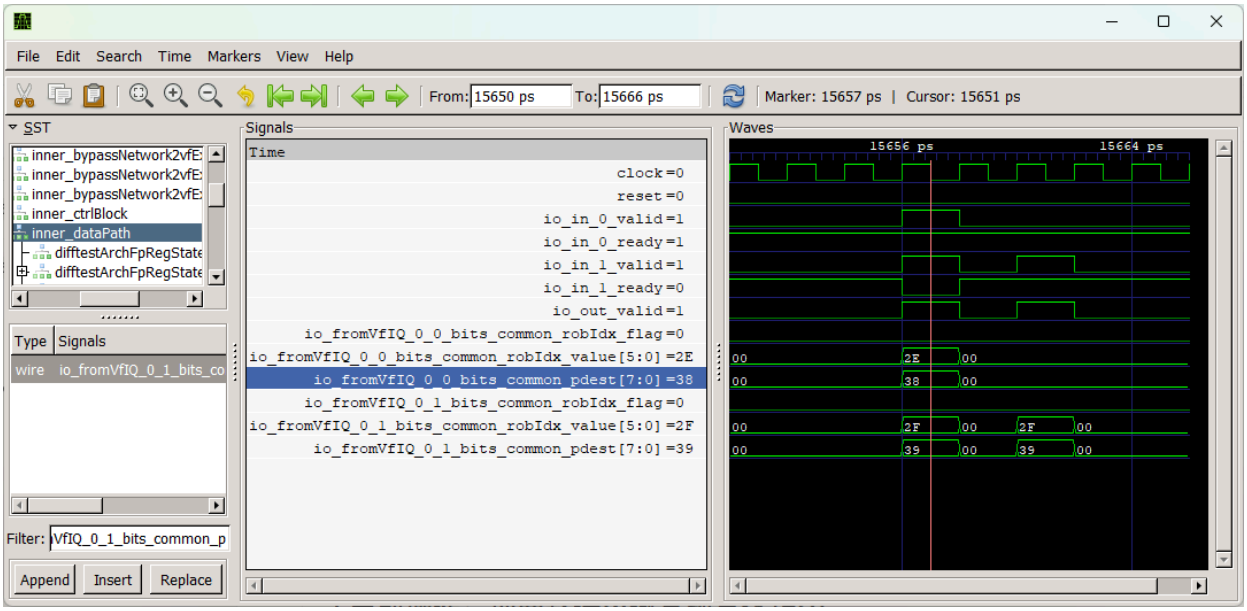


图 6-12　15658 ps 共同请求时，VDot 的 valid/ready 为 1/1，VIOTA 为 1/0，后者延后两周期获准。

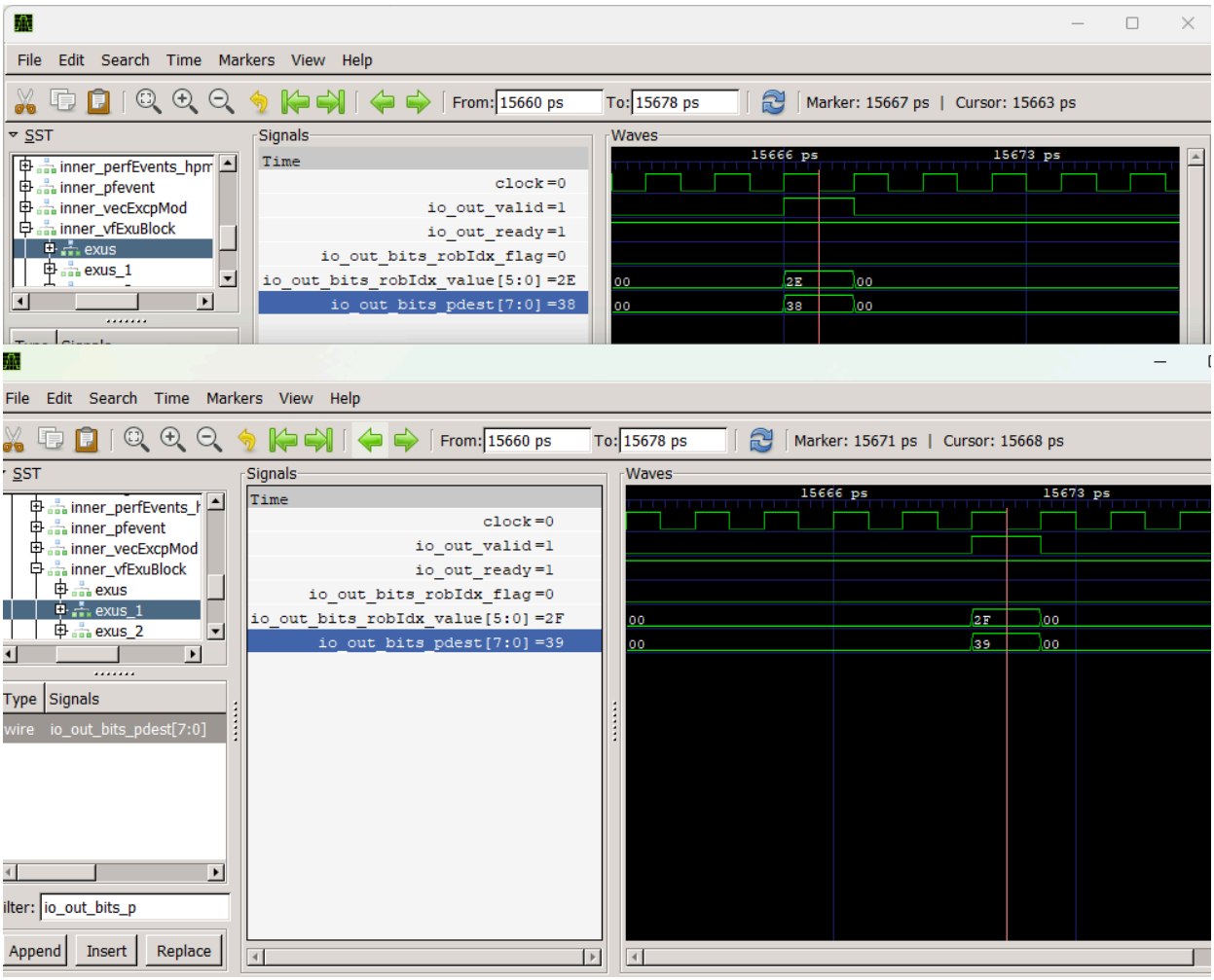


图 6-13　两条完成事件保持各自的 ROB/pdest，VDot 与 VIOTA 的结果在不同周期产生。

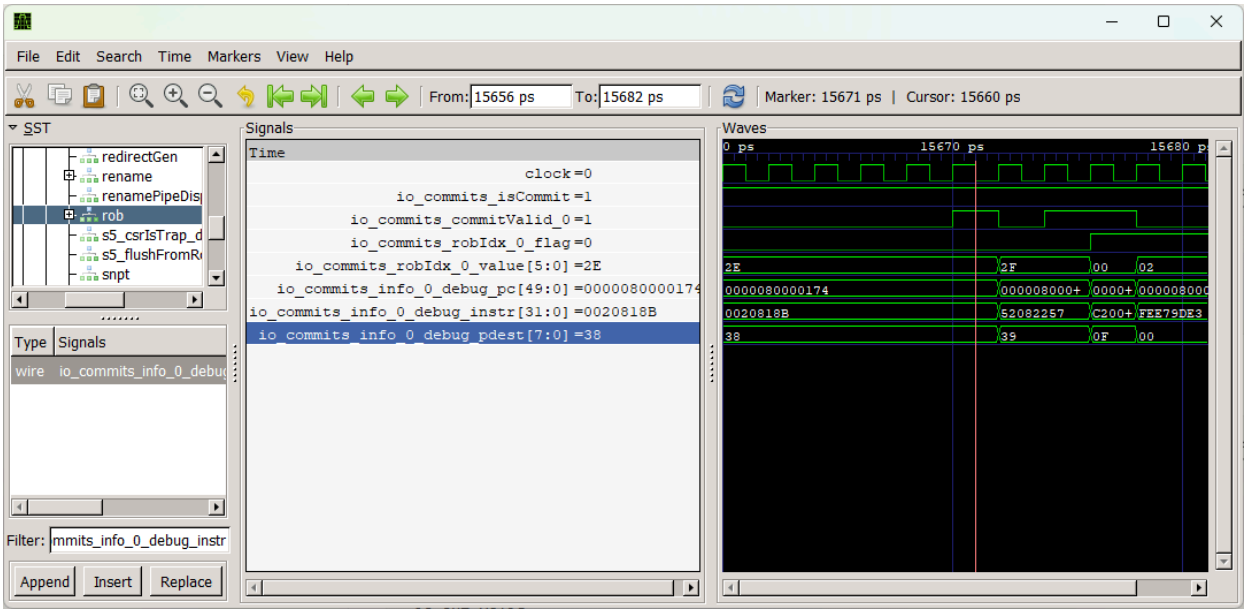


图 6-14　15671 ps 的 Marker 显示 VDot 退休沿前身份，后续时间窗显示 VIOTA 的退休身份。

VDot 和 VIOTA 争用的是发射前的写回端口预约。VDot 先获准，VIOTA 等两周期后获准，所以两条指令在不同周期写回并正常退休。本例记录的是预约冲突后的等待与推进；同拍结果写回和其他 FU 组合需另设用例〔15〕〔16〕〔17〕。

附录 B 源码与验证索引

本附录提供恢复方法与原件位置；正文只说明设计、关键源码和结果。第二阶段采用 SCNet Linux 服务器、单核 MinimalConfig、VLEN=128、ROB=48、EMU_TRACE=1。工具链记录包括 JDK 21、Mill 0.12.3、Verilator 5.052。

B.1 恢复源码和运行身份

XiangShan 基版本为 `16ae9ddcda54fc9a2fddffad73174cf793ac7814`。在独立 Linux 工程中恢复时，使用完整 RTL 补丁，并按 manifest 恢复指定依赖〔18〕〔19〕。完整补丁已包含首版接入与异常修复，不要再叠加首版补丁。

NEMU 基版本为 `a48a11f208ff81ad765591d83271d1b339fb3861`，使用 NEMU 补丁和保存的配置〔20〕。依次构建参考库、emu 和测试应用；在已有成功环境中重复测试时，只需按对应应用的需要构建。历史构建参数由构建脚本保存〔21〕。

服务器实验根为 `/root/private_data/mj/xiangshan`。修复版 emu 的相对路径为 `vdot-mtval-fix-20261024/XiangShan/build/emu`，源码参考库为 `vdot-dev-20261024/NEMU/build/riscv64-nemu-interpreter-so`，共同 Flash 为 `evidence/vdot-build-80mKCD/boot-flash.bin`。

表 B-1　成功运行保存的关键 SHA256

文件	SHA256
emu	`f9653532fd2a4b9f0d0cc08c131f7c258d4b62c46e672c6886d6643aad926850`
NEMU 参考库	`11b9f6807c36ed860e0c64cc4e58d27216b19335ecde1ce5797ba2d1cd057b74`
NEMU 配置	`64282bca051b5dbc5fca742f4fbca2ade937a70f3c4d2ec93c1f4836aca5d9be`
Flash	`fea548655a8b6eecacace83e2f4380a18b8aa83bce964c8aae4cffbe2f5a9724`

表 B-1 的校验值用于定位归档实验的仿真器、参考库、配置和 Flash。原始身份见运行文件校验清单〔22〕。

B.2 功能与动态事件材料

内容	工程入口
指令规格与参数	规格〔2〕、JSON 参数〔3〕
RTL 修改文件	源码快照〔23〕
正负数及边界程序	测试源文件目录〔24〕
基础、异常及重叠回归	修复回归记录〔12〕
正常与取消实例	原事件〔25〕、完整窗口核验〔13〕
退休到 CSR 请求	逐拍核验〔14〕
混发与路由	混发事件〔15〕、路由核对〔26〕
发射前端口竞争	端口事件〔16〕、退休身份补图〔17〕
完整事件及 VCD 归档	归档目录〔27〕

基础功能测试与代表性波形分别对应第 6 章列明的输入和窗口。未覆盖的取消时刻、实际结果写回同拍碰撞或其他资源组合，需要对应的独立用例。

材料完整路径

编号与本阶段正文一致。路径相对于配套 RISCV_vdot 工程根目录；图片随本文档目录提供，完整源码、日志和大型 VCD 由配套工程提供。

编号	材料	项目相对路径
1	编码检查	`docs/reviews/vdot-encoding-audit-2026-10-19.md`
2	指令规格基线	`docs/vdot-spec-v0.1.md`
3	机器可读参数	`specs/vdot-v0.1.json`
4	DecodeUnit 源码	`artifacts/vdot-mtval-fix-2026-10-24/patch-fixture-v2/src/main/scala/xiangshan/backend/decode/DecodeUnit.scala`
5	FuConfig 源码	`artifacts/vdot-mtval-fix-2026-10-24/patch-fixture-v2/src/main/scala/xiangshan/backend/fu/FuConfig.scala`
6	VDot.scala	`artifacts/vdot-mtval-fix-2026-10-24/patch-fixture-v2/src/main/scala/xiangshan/backend/fu/wrapper/VDot.scala`
7	VdotTrapInst.scala	`artifacts/vdot-mtval-fix-2026-10-24/patch-fixture-v2/src/main/scala/xiangshan/backend/rob/VdotTrapInst.scala`
8	CSR.scala	`artifacts/vdot-mtval-fix-2026-10-24/patch-fixture-v2/src/main/scala/xiangshan/backend/fu/wrapper/CSR.scala`
9	完整交付补丁	`artifacts/vdot-mtval-fix-2026-10-24/delivery-v2`
10	C 源码	`artifacts/vdot-integration-2026-10-24/delivery/payload/vdot-smoke/main.c`
11	汇编包装	`artifacts/vdot-integration-2026-10-24/delivery/payload/vdot-smoke/vdot.S`
12	修复版整核回归	`artifacts/vdot-mtval-fix-2026-10-24/server-xYGYlZ/review.json`
13	正常与取消窗口核验	`artifacts/vdot-window-supplement-2026-10-25/server-fgpWCe/review.json`

编号	材料	项目相对路径
14	CSR 请求核验	artifacts/vdot-csr-commit-window-2026-10-25/server-aReAEz/review.json
15	混发分析	artifacts/vdot-mixed-resources-2026-10-25/server-9k3waS/review.json
16	端口事件	artifacts/vdot-port-events-2026-10-25/server-FfVozt/review.json
17	退休补图	artifacts/vdot-gtk-review-2026-10-26/step3-retirement-instruction/review.json
18	完整 RTL 补丁	artifacts/vdot-mtval-fix-2026-10-24/delivery-v2/XiangShan-full.patch
19	manifest	artifacts/vdot-mtval-fix-2026-10-24/delivery-v2/manifest.json
20	NEMU 补丁	artifacts/vdot-integration-2026-10-24/delivery/payload/NEMU.patch
21	构建脚本	artifacts/vdot-mtval-fix-2026-10-24/delivery-v2/build_scnet_vdot_mtval.sh
22	运行文件校验清单	artifacts/vdot-mtval-fix-2026-10-24/server-xYGYlZ/accepted-identities.sha256
23	源码快照	artifacts/vdot-mtval-fix-2026-10-24/patch-fixture-v2
24	测试源文件目录	artifacts/vdot-integration-2026-10-24/delivery/payload/vdot-smoke
25	原事件	artifacts/vdot-window-events-2026-10-25/server-UbYWTP/review.json
26	路由核对	artifacts/vdot-mixed-route-audit-2026-10-25/server-ApJqcC/review.json
27	归档目录	artifacts/stage12-archive-audit-2026-10-27/server-LpAu2Y